

Quiz 3 - Solutions

This is an individual assignment.

Write your answers using markdown cells or embed handwritten answers with `IPython.display.Image`.

Consider the following dataset

x1	x2	t
1	0	1
4	2	1
0	-1	-1
-1	-1	-1
-2	1	-1

where t is the ground truth target label. Answer the following questions:

Exercise 1 (3 points)

Consider the *Fisher's LDA classifier* and answer the following sub-parts:

1. (1.5 points) **Compute the between-class scatter matrix, S_B , and the within-class scatter matrix, S_W .**
2. (1.5 points) **Find w and w_0 . Draw the respective discriminant function.**

Let's compute S_B and S_W :

```
In [1]: import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline
plt.style.use('bmh')
```

```
In [2]: # Dataset
X = np.array([[1,0],[4,2],[0,-1],[-1,-1],[-2,1]])
t = np.array([1,1,-1,-1,-1])

# Means
m1=np.mean(X[t==1,:],axis=0).reshape(-1,1)
m2=np.mean(X[t==-1,:],axis=0).reshape(-1,1)
```

```

# Between-class scatter matrix
SB=(m1-m2)@(m1-m2).T
print('SB = ', SB)

# Within-class scatter matrix
SW = (X[t==1,:]-m1.T).T@(X[t==1,:]-m1.T) + (X[t==1,:]-m2.T).T@(X[t==1,:]-m2.T)
print('SW = ', SW)

```

```

SB = [[12.25      4.66666667]
      [ 4.66666667  1.77777778]]
SW = [[6.5      1.      ]
      [1.      4.66666667]]

```

2. The projection vector, $\mathbf{w}$, is the eigenvector of $S_W^{-1}S_B$ with the largest eigenvalue, and $w_0 = -\frac{\mathbf{w}^T(\mathbf{m}_1+\mathbf{m}_2)}{2}$.

```

In [3]: # Eigendecomposition
L,E = np.linalg.eigh(np.linalg.inv(SW)@SB)
w = E[:,np.argmax(L)]
w0 = -(m1+m2).T@w/2

print('w0 = ', w0)
print('w = ', w)

```

```

w0 = [0.81791171]
w = [-0.94434151 -0.32896674]

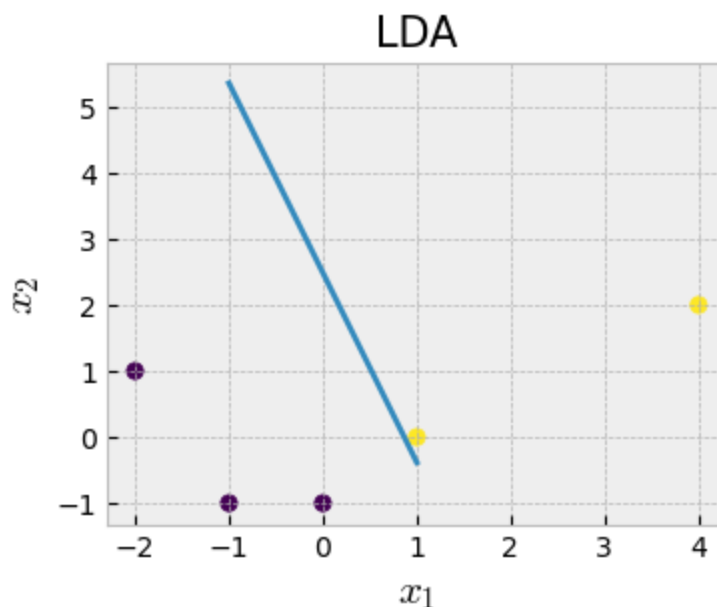
```

```

In [4]: x_line = np.linspace(-1,1,100)
y_line = -(w[0]/w[1])*x_line - w0/w[1]

plt.figure(figsize=(4,3)); plt.scatter(X[:,0], X[:,1],c=t); plt.plot(x_line,
plt.xlabel('$x_1$',size=15); plt.ylabel('$x_2$',size=15); plt.title('LDA',si

```



Exercise 2 (3 points)

Consider the Perceptron classifier with the initial weights $w = [4, 1]$ and intercept $w_0 = 2$. Answer the following sub-parts:

1. (1.5 points) Draw the samples and the corresponding discriminant function.
2. (1.5 points) What is the smallest value for the learning rate η such that the updated Perceptron will result in zero misclassified points using only one iteration?

1. The discriminant function can be written as

$$y(\mathbf{x}) = \phi(\mathbf{w}^T \mathbf{x} + w_0) = \phi(4x_1 + x_2 + 2)$$

where $\phi(x) = \begin{cases} 1, & x > 0 \\ -1, & x \leq 0 \end{cases}$ is the threshold function.

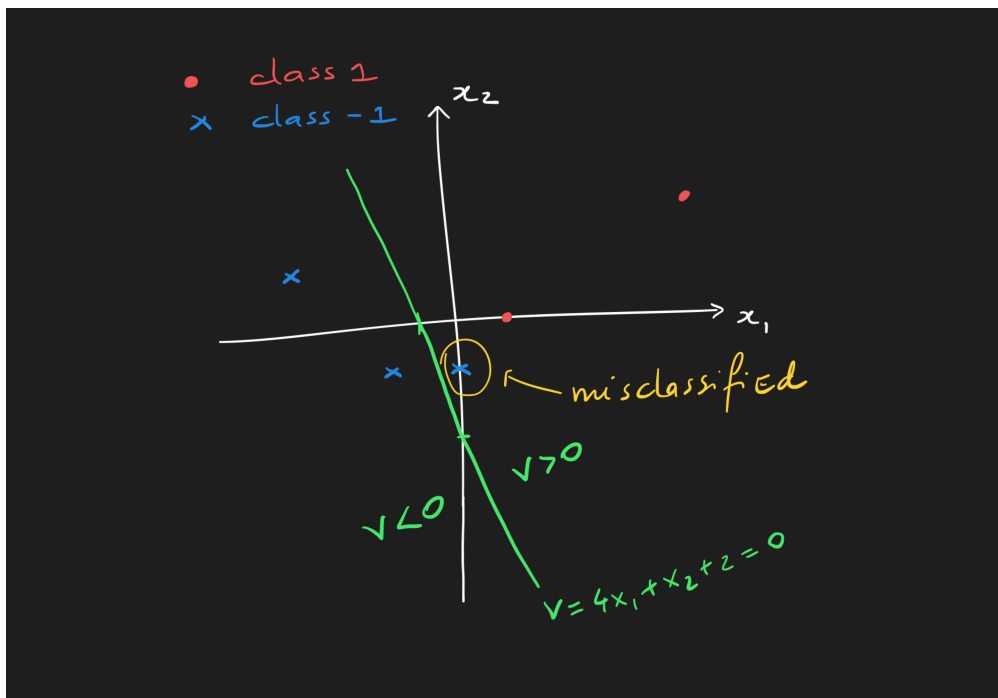
We classify as 1 if $y(\mathbf{x}) > 0 \iff 4x_1 + x_2 + 2 > 0$, otherwise classify as 0.

we can rewrite the discriminant function as:

$$x_2 = -4x_1 - 2$$

```
In [5]: from IPython.display import Image
Image('figures/discriminant_function.jpg',
      width=500)
```

Out [5]:



2. Let's evaluate the output for each of the 5 data samples provided:

x	y	z	t
1	0	1	1
4	2	1	1
0	-1	1	-1
-1	-1	-1	-1
-2	1	-1	-1

As the Perceptron algorithm uses misclassified points to make updates in its parameters (weights and bias), the first step is to identify if there are any misclassified points.

For the table above, we can see that data sample $(x_3, y_3) = (0, -1)$ is misclassified. So this will be the only point that will be used to make the updates for the weights and bias term of the Perceptron.

The update equations are given as:

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} + \eta \mathbf{x}_n t_n = \begin{bmatrix} 4 \\ 1 \end{bmatrix} + \eta \begin{bmatrix} 0 \\ -1 \end{bmatrix} (-1) = \begin{bmatrix} 4 \\ 1 + \eta \end{bmatrix}$$

$$w_0^{(t+1)} = w_0^{(t)} + \eta t_n = 2 + \eta(-1) = 2 - \eta$$

The new equation for the decision surface is:

$$z = \phi(v), \text{ where } v = 4x + (1 + \eta)y + 2 - \eta$$

Putting it all together for the misclassified point, we now have:

$$\begin{aligned} v &= \mathbf{w}^{(t+1)^T} \begin{bmatrix} 0 \\ -1 \end{bmatrix} + b^{(t+1)} \\ &= [4, 1 + \eta] \begin{bmatrix} 0 \\ -1 \end{bmatrix} + 2 - \eta \\ &= -1 - \eta + 2 - \eta \\ &= 1 - 2\eta \end{aligned}$$

Now, note that, from the activation function we are using, the point $[0, -1]$ will be classified with label -1 if $v \leq 0$, hence:

$$1 - 2\eta \leq 0 \iff \eta \geq 0.5$$

The smallest value for the learning rate such that the updated network will result in zero misclassified points using only one iteration is $\eta = 0.5$.

Check:

$$\eta = 0.5 : z = \phi(v), \text{ where } v = 4x + 1.5y + 1.5$$

x	y	v	z	t
1	0	5.5	1	1
4	2	20.5	1	1
0	-1	0	-1	-1
-1	-1	-4	-1	-1
-2	1	-5	-1	-1

```
In [6]: # Dataset
X = np.array([[1,0],[4,2],[0,-1],[-1,-1],[-2,1]])
t = np.array([1,1,-1,-1,-1])

# Initial parameters
w = np.array([[4,1]]).T
w0 = 2

# weighted sum
v = X@w+w0

# prediction
y = -1*np.ones((len(t),1))
y[v>0] = 1
y
```

```
Out[6]: array([[ 1.],
               [ 1.],
               [ 1.],
               [-1.],
               [-1.]])
```

```
In [7]: eta = 0.1

w = (w.T + eta*t[2]*X[2,:]).T
w0 = w0-eta

# weighted sum
v = X@w+w0

# prediction
y = -1*np.ones((len(t),1))
y[v>0] = 1
y
```

```
Out[7]: array([[ 1.],
               [ 1.],
               [ 1.],
               [-1.],
               [-1.]])
```

```
In [8]: eta = 0.1

w = (w.T + eta*t[2]*X[2,:]).T
```

```

w0 = w0-eta

# weighted sum
v = X@w+w0

# prediction
y = -1*np.ones((len(t),1))
y[v>0] = 1
y

```

```

Out[8]: array([[ 1.],
               [ 1.],
               [ 1.],
               [-1.],
               [-1.]])

```

```

In [9]: w, w0

```

```

Out[9]: (array([[4. ],
               [1.2]]),
        1.7999999999999998)

```

Exercise 3 (3 points)

Consider the Logistic Regression classifier with the initial weights $w = [4, 1]$ and intercept $w_0 = 2$. Let a label $t = -1$ be mapped to $t = 0$. Answer the following sub-parts:

- (1.5 points) Use the initial parameter values to predict the label for each samples above.
- (1.5 points) What is the smallest value for the learning rate η such that the updated Logistic Regression will result in zero misclassified points using only one iteration?

- The discriminant function is of the form

$$y(\mathbf{x}) = \phi(\mathbf{w}^T \mathbf{x} + w_0) = \phi(4x_1 + x_2 + 2)$$

where $\phi(x) = \frac{1}{1+e^{-x}}$ is the logistic/sigmoid activation function. In the table below, consider $v = \mathbf{w}^T \mathbf{x} + w_0 = 4x_1 + x_2 + 2$, thus $y(\mathbf{x}) = \phi(v)$.

x1	x2	v	y	t
1	0	6	0.998	1
4	2	20	1	1
0	-1	1	0.731	0
-1	-1	-3	0.047	0

x1	x2	v	y	t
-2	1	-5	0.007	0

```
In [10]: # Dataset
X = np.array([[1,0],[4,2],[0,-1],[-1,-1],[-2,1]])
t = np.array([[1],[1],[0],[0],[0]])

X.shape, t.shape
```

```
Out[10]: ((5, 2), (5, 1))
```

```
In [11]: # Logistic function
logistic = lambda x: 1/(1+np.exp(-x))

# Initial parameters
w = np.array([[4,1]]).T
w0 = 2

# weighted sum
v = X@w+w0

# Predictions (probability of assigning to class 1)
prob = logistic(v)

# If prob>=0.5, then classify as 1, otherwise, classify as 0
y = 1*(prob>=0.5)

v, prob, y
```

```
Out[11]: (array([[ 6],
                [20],
                [ 1],
                [-3],
                [-5]]),
          array([[0.99752738],
                [1.],
                [0.73105858],
                [0.04742587],
                [0.00669285]]),
          array([[1],
                [1],
                [1],
                [0],
                [0]]))
```

We can see that the sample $[0, -1]$ is misclassified, because logistic regression outputs a probability larger than 0.5, which would assign it to class 1.

2. The update equations are given as:

$$\begin{aligned}\mathbf{w}^{(t+1)} &= \mathbf{w}^{(t)} + \eta \sum_{i=1}^N (t_i - \phi(v_i)) \mathbf{x}_i \\ &= \begin{bmatrix} 4 \\ 1 \end{bmatrix} + \eta((1 - 0.998) \begin{bmatrix} 1 \\ 0 \end{bmatrix} + (1 - 1) \begin{bmatrix} 4 \\ 2 \end{bmatrix} + \dots \\ &\quad + (1 - 0.731) \begin{bmatrix} 0 \\ -1 \end{bmatrix} + (0 - 0.047) \begin{bmatrix} -1 \\ -1 \end{bmatrix} + (0 - 0.007) \begin{bmatrix} -2 \\ 1 \end{bmatrix})\end{aligned}$$

and

$$\begin{aligned}w_0^{(t+1)} &= w_0^{(t)} + \eta \sum_{i=1}^N (t_i - \phi(v_i)) \\ &= 2 + \eta((1 - 0.998) + (1 - 1) + (1 - 0.731) + (0 - 0.047) + (0 - 0.007))\end{aligned}$$

```
In [12]: np.sum(t*np.log(logistic(X@w+w0)) + (1-t)*np.log(1-logistic(X@w+w0)))
```

```
Out[12]: -1.371040074779967
```

```
In [13]: (1-t[2])*np.log(1-logistic(X[2,:@w+w0]))
```

```
Out[13]: array([-1.31326169])
```

```
In [14]: w.T+0.1*(t[2]-logistic(X[2,:@w+w0]))*X[2,:]
```

```
Out[14]: array([[4.          , 1.07310586]])
```

```
In [15]: # Calculations for w
X.T@t-prob
```

```
Out[15]: array([[0.06328421],
               [0.77179161]])
```

```
In [16]: # Calculations for w0
np.sum(t-prob)
```

```
Out[16]: -0.7827046775140681
```

We find:

$$\mathbf{w}^{(t+1)} = \begin{bmatrix} 4 + 0.06328421\eta \\ 1 + 0.77179161\eta \end{bmatrix}$$

and

$$w_0^{(t+1)} = 2 - 0.7827046775140681\eta$$

The new equation for the discriminant function is:

$$y(\mathbf{x}) = \phi(v), \text{ where } v = \begin{bmatrix} 4 + 0.06328421\eta \\ 1 + 0.77179161\eta \end{bmatrix}^T \mathbf{x} + 2 - 0.7827046775140681\eta$$

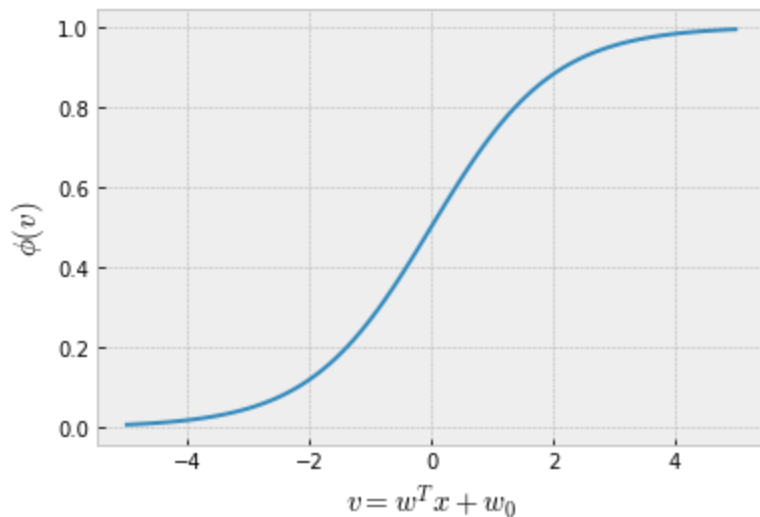
Putting it all together for the misclassified point, we now have:

$$\begin{aligned} v &= \begin{bmatrix} 4 + 0.06328421\eta \\ 1 + 0.77179161\eta \end{bmatrix}^T \begin{bmatrix} 0 \\ -1 \end{bmatrix} + 2 - 0.7827046775140681\eta \\ &= -1 - 0.77179161\eta + 2 - 0.7827046775140681\eta \\ &= 1 - (0.77179161 + 0.7827046775140681)\eta \\ &= 1 - 1.554496287514068\eta \end{aligned}$$

In [17]: `0.77179161+0.7827046775140681`

Out[17]: `1.554496287514068`

In [18]: `xline = np.linspace(-5,5,100)
plt.plot(xline, logistic(xline))
plt.xlabel('$v=w^Tx + w_0$', size=15)
plt.ylabel('$\phi(v)$', size=15);`



Now, note that, from the activation function we are using, the point $[0, -1]$ will be classified with label 0 if $\phi(v) < 0.5 \iff v < 0$.

$$1 - 1.554496287514068\eta < 0 \iff \eta > 0.6432951998870245 \Rightarrow \eta = 0.65$$

In [19]: `1/1.554496287514068`

Out[19]: `0.6432951998870245`

You can achieve this in the same way, if you consider:

$$\begin{aligned}\phi(\mathbf{w}^{(t+1)^T} \mathbf{x} + w_0^{(t+1)}) &< 0.5 \\ \mathbf{w}^{(t+1)^T} \mathbf{x} + w_0^{(t+1)} &< \phi^{-1}(0.5) \\ 1 - 1.554496287514068\eta &< 0 \\ \eta &> 0.6432951998870245 \\ \Rightarrow \eta &= 0.65\end{aligned}$$

You can find the inverse of $\phi(x)$ like this:

$$\frac{1}{1 + e^{-x}} = y \iff \frac{1}{y} - 1 = e^{-x} \iff x = -\ln\left(\frac{1}{y} - 1\right) \implies \phi^{-1}(x) = -\ln\left(\frac{1}{x} - 1\right)$$

```
In [20]: # Inverse logistic
logistic_inv = lambda x: -np.log(1/x - 1)

logistic_inv(0.5)
```

```
Out[20]: -0.0
```

```
In [21]: # learning rate
eta = 0.65
```

```
In [22]: # Updating w
new_w = w + eta*X.T@(t-prob)

new_w
```

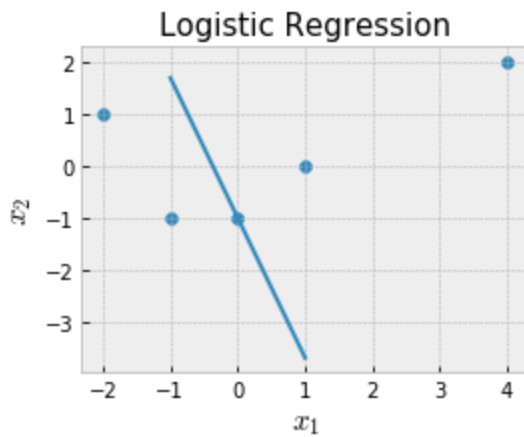
```
Out[22]: array([[4.04113473],
               [1.50166454]])
```

```
In [23]: # Updating w0
new_w0 = 2 + np.sum(t-prob)*eta
new_w0
```

```
Out[23]: 1.4912419596158557
```

```
In [24]: x_line = np.linspace(-1,1,100)
y_line = -x_line*new_w[0]/new_w[1] - new_w0/new_w[1]

plt.figure(figsize=(4,3)); plt.scatter(X[:,0], X[:,1],c=t); plt.plot(x_line,
plt.xlabel('$x_1$',size=15); plt.ylabel('$x_2$',size=15); plt.title('Logisti
```



```
In [25]: # Check

# weighted sum
v_new = X@new_w+new_w0

# Predictions (probability of assigning to class 1)
prob = logistic(v_new)

# If prob>=0.5, then classify as 1, otherwise, classify as 0
y = 1*(prob>=0.5)
y
```

```
Out[25]: array([[1],
                [1],
                [0],
                [0],
                [0]])
```

Note that this is the **smallest** learning rate need to correctly classify all sample, but it will not maximize the cross-entropy.

On-Time (1 point)

Submit your assignment before the deadline.

Submit Your Solution

Confirm that you've successfully completed the assignment.

Along with the Notebook, include a PDF of the notebook with your solutions.

`add` and `commit` the final version of your work, and `push` your code to your GitHub repository.

Submit the URL of your GitHub Repository as your assignment submission on Canvas.
